

Week 1 Progress Report

Multimodal AI E-commerce
Intelligence Automation Agent

Backend Foundation · Weeks 1 of N

Status: **Completed** ✓

FastAPI	Qdrant	Docker	Ollama VLM	Pillow	Python
---------	--------	--------	------------	--------	--------

1. Project Overview

The Week 1 goal was to build the backend foundation for a Multimodal AI E-commerce Intelligence Automation Agent. By the end of the week, the project reached a working prototype where a user can upload a product image, have it analysed by a Vision Language Model, and retrieve similar products from a vector database.

Core Pipeline

```
Image Upload
→ VLM Product Analysis
→ Product Metadata Extraction
→ Search Text Generation
→ Embedding Generation
→ Qdrant Vector Search
→ Similar Product Results
```

2. Day-by-Day Summary

Day 1 – Project Setup

Created the GitHub repository, planned the folder structure, selected FastAPI as the backend framework, and discussed high-level architecture including potential future use of fine-tuning / LoRA.

Key Concepts Learned

- GitHub repo & local project folder
- Backend-first development approach
- API planning
- Basic fine-tuning & LoRA concepts

Day 2 – FastAPI Backend

Built the FastAPI application, added basic routes, and connected the project to GitHub. The backend became runnable via Uvicorn with Swagger docs at <http://127.0.0.1:8000/docs>.

Key Concepts Learned

- FastAPI app structure & Uvicorn
- Swagger documentation
- Python package imports
- Git merge-conflict handling

Day 3 – Qdrant Integration

Added Qdrant as the vector database (via Docker). Created product ingestion and search APIs. The first working semantic product search was demonstrated.

Key Concepts Learned

- Qdrant vector database & Docker
- Embeddings & semantic search
- Product ingestion & search API

Day 4 – Vision Language Model

Integrated the VLM (qwen2.5vl:3b via Ollama) to analyse product images. Structured JSON output was extracted from model responses.

Key Concepts Learned

- VLM image-to-text analysis
- Ollama local model serving
- Base64 image encoding
- Prompt engineering

Day 5 – VLM + Product Save Flow

Connected image analysis to product creation. Uploading an image now automatically generates a product name and description, which is saved into Qdrant.

Key Concepts Learned

- Combining AI output with backend logic
- Auto-saving AI-generated product data

Day 6 – Proposal & Business Case

Wrote the project proposal and articulated why a specialised SaaS product has value compared to general-purpose AI assistants.

Key Concepts Learned

- SaaS positioning
- Local vs cloud VLM trade-offs
- MCP server vs FastAPI discussion

Day 7 – Full Pipeline Testing

Integration day: tested all endpoints end-to-end, fixed import issues, optimised VLM performance, improved prompt quality, and hardened output parsing.

Key Concepts Learned

- Image compression (Pillow)
- VLM prompt improvement
- `safe_join()` / `safe_text()` robust parsing
- Score-filtered search results

3. Technical Deep-Dives

3.1 Backend Startup Debugging

Two import-path errors were encountered and resolved:

```
# Error 1
ModuleNotFoundError: No module named 'backend'

# Error 2
ModuleNotFoundError: No module named 'app'

# Fix – run from inside the backend folder
cd "C:\Users\...\backend"
uvicorn app.main:app --reload
```

3.2 VLM Performance Optimisation

VLM inference ran entirely on CPU, causing very long response times. Several techniques were applied to reduce latency:

- Image compression: resize to 512 px max side, convert to JPEG
- Reduced context window: num_ctx 4096 → 2048
- Limited output length with num_predict
- Added keep_alive to keep model loaded between requests
- Improved prompt to focus on the product, not background

Stage	Response Time
Initial (no optimisation)	~3 min 52 sec
After first run (warm model)	~3 min 30 sec
After full optimisation	~1 min 49 sec
Image search (optimised)	~1 min

3.3 Robustness Fix – Mixed VLM Output

The VLM occasionally returned dictionaries inside lists for visible_features. Safe conversion helpers were added to prevent crashes:

```
# Problem
[{"feature": "Shape", "description": "Round and saddle-like"}]

# After safe_text() / safe_join()
"Shape Round and saddle-like"
```

4. Search Results

4.1 Text Search

```
Query: "brown formal slim fit shirt"
Top result: Male Brown Slim Fit Shirt | score: 0.859 | Formal Wear
```

4.2 Image Search – Shirt

```
Endpoint: POST /api/v1/multimodal/search/image
Top result: Male Brown Slim Fit Shirt | score: 0.905 | Formal Wear
```

4.3 Image Search – Handbag

```
Input: Brown saddle handbag photograph
VLM: product_type=Handbag, style=Saddle Bag, material=Leather
Top result: Unisex Brown Saddle Bag Handbag | score: 0.924 | Accessories
```

5. API Reference

Method	Endpoint	Description
--------	----------	-------------

GET	/api/v1/products/	List all products
POST	/api/v1/products/	Create a product manually
POST	/api/v1/products/search	Text-based semantic search
POST	/api/v1/products/analyze-and-save	Upload image, analyse & save
GET	/api/v1/multimodal/search/text	Multimodal text search
POST	/api/v1/multimodal/search/image	Multimodal image search

6. Tech Stack

Component	Technology
Backend Framework	FastAPI
Server	Uvicorn
Vector Database	Qdrant
Container Runtime	Docker
Vision LLM	qwen2.5vl:3b via Ollama
Image Processing	Pillow
API Testing	Swagger UI
Version Control	Git & GitHub

7. Problems Faced & Debugging

- Python import path errors → fixed by running Uvicorn from inside /backend
- Qdrant container management → verified with docker ps
- VLM slow inference on CPU → resolved with image compression & context reduction
- Missing Pillow dependency → pip install pillow + requirements.txt update
- VLM including background scene details → improved prompt engineering
- VLM returning dicts inside lists → added safe_join() / safe_text() helpers
- Weak low-score search results → added score thresholding
- Duplicate products from repeated testing → flagged for Week 2

8. Final Status & Week 2 Plan

Feature	Status
FastAPI backend	✓ Working
Qdrant vector DB	✓ Working
Text search	✓ Working
Image search	✓ Working
VLM analysis	■ Working (CPU-slow)

Frontend	✗ Not started
Authentication	✗ Not started
Product CRUD	■ Partial (no delete/update)

Week 2 Recommended Focus

- Build React / Next.js frontend with Tailwind CSS
- Connect frontend to FastAPI via Axios
- Product upload form + image search UI
- Loading states for slow VLM operations
- Product duplicate detection & removal
- Improve API response formatting & error handling
- Add score threshold controls in UI
- Prepare project demo flow